

UNIT- II

Relational Model

- Introduction to relational model
- Concepts of domain
- Attribute
- Tuples
- Relation
- Importance of null values

The Relational Database: Definitions

- A DBMS that manages data as collection of tables in which all data relationships are represented by common values in related tables.”
- “A DBMS that follows all the twelve rules of CODD is called RDBMS”.

All information must be represented explicitly in one and only one way: as values in tables and each & every datum in the database must be accessible by specifying a table name, a column name, and a primary key

Relational Model

- **Relational Model (RM)** represents the database as a collection of relations. A relation is nothing but a table of values. Every row in the table represents a collection of related data values.
- The Relational Model developed by Dr. E. F. Codd at IBM in the late 1960s.
- The model built on mathematical concepts, which expounded in the famous work called "A Relational Model of Data for Large Shared Databanks ".
- At the core of the relational model is the concept of a table (also called a relation) in which all data is stored.
- Records (horizontal rows also known as tuples) & Fields (vertical columns also known as attributes).
- Table can be identified by a unique name.

A table is a collection of records and each record in a table contains the same fields

Properties of the Relational model

- Data is presented as a collection of relations.
- Each relation is depicted as a table.
- Columns are attributes that belong to the entity modelled by the table (ex. In a student table, you could have name, address, student ID, major, etc.).

- Each row ("tuple") represents a single entity (ex. In a student table, John , NRT, 12345, Accounting, would represent one student entity).
- Every table has a set of attributes that taken together as a "**key**" (technically, a "superkey")
- Uniquely identifies each entity (Ex. In the student table, "student ID" would uniquely identify each student – no two students would have the same student ID).

Relational Model Concepts

Attribute: Each column in a Table. Attributes are the properties which define a relation. e.g., Student_Rollno, NAME,etc.

Tables : In the Relational model the, relations are saved in the table format. It is stored along with its entities. A table has two properties rows and columns. Rows represent records and columns represent attributes.

Tuple: It is nothing but a single row of a table, which contains a single record.

Relation Schema: A relation schema represents the name of the relation with its attributes.

Degree: The total number of attributes which in the relation is called the degree of the relation.

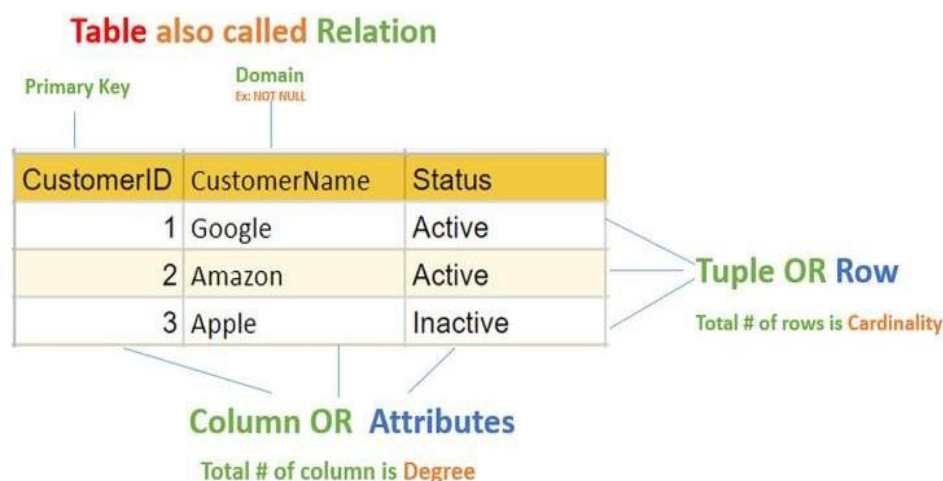
Cardinality: Total number of rows present in the Table.

Column: The column represents the set of values for a specific attribute.

Relation instance – Relation instance is a finite set of tuples in the RDBMS system. Relation instances never have duplicate tuples.

Relation key - Every row has one, two or multiple attributes, which is called relation key.

Attribute domain – Every attribute has some pre-defined value and scope which is known as attribute domain.



PRINCIPLES OF RELATIONAL MODEL

- Data about various entities and their relationships are stored in a series of logical tables (also known as relations).

- A relation is a two-dimensional table with certain imposed restrictions:

1. Each Row is unique: No duplicate row
2. Entries in any column have the same domain.
3. Each column has a unique name
4. Order of the columns or rows is irrelevant
5. Each entry in the table is single valued:
No group item, repeating group, or array is allowed.

Relation: made up of 2 parts:

- *Instance* (set of records): a *table*, with rows and columns.
#Rows = *cardinality*, #fields = *degree / arity*.
 - *Schema* : specifies name of relation, plus name and domain (type) of each field (column).
 - E.G. Students(*sid*: string, *name*: string, *login*: string, *age*: integer, *gpa*: real).
- Example Instance of Students Relation

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

Cardinality = 3, degree = 5, all rows distinct

- ❖ Do all columns in a relation instance have to be distinct?

Domain constraints in DBMS

- ❖ A table in DBMS is a set of rows and columns that contain data.
- ❖ Columns in table have a unique name, often referred as attributes in DBMS.
- ❖ A domain is a unique set of values permitted for an attribute in a table.

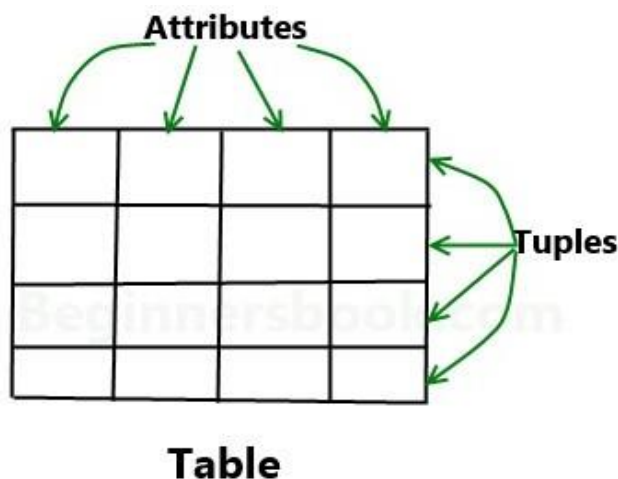
- ❖ For example, a domain of month-of-year can accept January, February....December as possible values,
- ❖ A domain of integers can accept whole numbers that are negative, positive and zero.
- ❖ **Definition:** Domain constraints are **user defined data type** and we can define them like this:
- ❖ Domain Constraint = data type + Constraints (NOT NULL / UNIQUE / PRIMARY KEY / FOREIGN KEY / CHECK / DEFAULT)
- ❖ **Example:**
For example i want to create a table "student_info" with "stu_id" field having value greater than 100, I can create a domain and table like this:
- ❖ **create domain id_value int constraint id_test check(value > 100);**
- ❖ **create table student_info (stu_id id_value PRIMARY KEY, stu_name varchar(30), stu_age int);**
- ❖ **Another example:**
I want to create a table "bank_account" with "account_type" field having value either "checking" or "saving":

create domain account_type char(12) constraint acc_type_test check(value in ("Checking", "Saving"));

create table bank_account (account_nbr int PRIMARY KEY, account_holder_name varchar(30), account_type account_type);

Attributes in DBMS

- In RDBMS, a table organizes data in rows and columns. The columns are known as attributes where as the rows are known as records.



Example: A school maintains the data of students in a table named "student". Suppose the data they store in table is student id, student name & student age. To do this they have had three columns in the table: **student_id**, **student_age**, **student_name**. The table looks like this:

student_id	student_age	student_name
101	12	Jon
102	13	Arya
103	12	Sansa

Here student_id, student_age and student_name are the **attributes**

Importance of null values

- A **NULL** value in a table is a value in a field that appears to be blank.
- A field with a **NULL** value is a field with no value.
- It is very important to understand that a NULL value is different than a zero value or a field that contains spaces.

Syntax

The basic syntax of **NULL** while creating a table.

```
CREATE TABLE CUSTOMERS
```

```
(
```

```
  ID INT NOT NULL,
```

```
  NAME VARCHAR (20) NOT NULL,
```

```
  AGE INT NOT NULL,
```

```
  ADDRESS CHAR (25) ,
```

```
  SALARY DECIMAL (18, 2),
```

```
  PRIMAREY (ID)
```

Here, **NOT NULL** signifies that column should always accept an explicit value of the given data type. There are two columns where we did not use NOT NULL, which means these columns could be **NULL**.

A field with a **NULL** value is the one that has been left blank during the record creation.

Example

- The NULL value can cause problems when selecting data.
- However, because when comparing an unknown value to any other value, the result is always unknown and not included in the results.
- You must use the **IS NULL** or **IS NOT NULL** operators to check for a NULL value.
- Consider the following CUSTOMERS table having the records as shown below.

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	
7	Muffy	24	Indore	

Now, following is the usage of the **IS NOT NULL** operator.

```
SQL> SELECT ID, NAME, AGE, ADDRESS, SALARY
      FROM CUSTOMERS
      WHERE SALARY IS NOT NULL;
```

This would produce the following result –

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00

Now, following is the usage of the **IS NULL** operator.

```
SQL> SELECT ID, NAME, AGE, ADDRESS, SALARY
      FROM CUSTOMERS
      WHERE SALARY IS NULL;
```

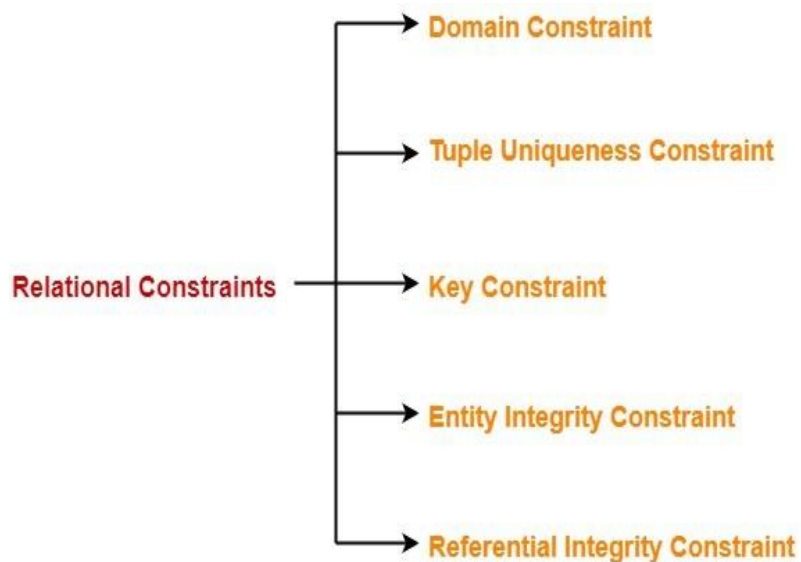
This would produce the following result –

ID	NAME	AGE	ADDRESS	SALARY
6	Komal	22	MP	
7	Muffy	24	Indore	

Constraints in DBMS

- Constraints or nothing but the rules that are to be followed while entering data into columns of the database table.
- Constraints ensure that data entered by the user into columns must be within the criteria specified by the condition.
- For example, if you want to maintain only unique IDs in the employee table or if you want to enter only age under 18 in the student table etc.

Types of Constraints in DBMS



Domain Constraint

- Domain constraint defines the domain or set of values for an attribute.
- It specifies that the value taken by the attribute must be the atomic value from its domain.
- **Example-** Consider the following Student table

STU_ID	Name	Age
S001	Akshay	20
S002	Abhishek	21
S003	Shashank	20
S004	Rahul	A

Here, value 'A' is not allowed since only integer values can be taken by the age attribute.

Tuple Uniqueness Constraint

- Tuple Uniqueness constraint specifies that all the tuples must be necessarily unique in any relation.

Example-01: Consider the following Student table

<u>STU_ID</u>	Name	Age
S001	Akshay	20
S002	Abhishek	21
S003	Shashank	20
S004	Rahul	20

This relation satisfies the tuple uniqueness constraint since here all the tuples are unique.

Example-02: Consider the following Student table

<u>STU_ID</u>	Name	Age
S001	Akshay	20
S001	Akshay	20
S003	Shashank	20
S004	Rahul	20

This relation does not satisfy the tuple uniqueness constraint since here all the tuples are not unique.

Key Constraints or Uniqueness Constraints

- These are called uniqueness constraints since it ensures that every tuple in the relation should be unique.
- A relation can have multiple keys or candidate keys(minimal superkey), out of which we choose one of the keys as primary key, we don't have any restriction on choosing the primary key out of candidate keys, but it is suggested to go with the candidate key with less number of attributes.

- Null values are not allowed in the primary key, hence Not Null constraint is also a part of key constraint.

Example- Consider the following Student table-

<u>STU_ID</u>	Name	Age
S001	Akshay	20
S001	Abhishek	21
S003	Shashank	20
S004	Rahul	20

This relation does not satisfy the key constraint as here all the values of primary key are not unique.

Entity Integrity Constraint

- Entity integrity constraint specifies that no attribute of primary key must contain a null value in any relation.
- This is because the presence of null value in the primary key violates the uniqueness property.

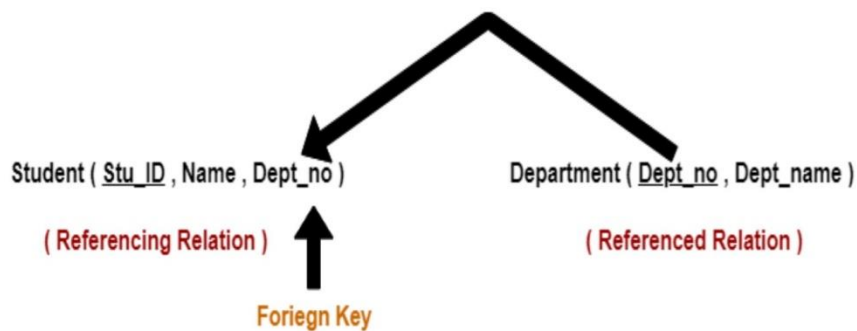
Example- Consider the following Student table-

<u>STU_ID</u>	Name	Age
S001	Akshay	20
S002	Abhishek	21
S003	Shashank	20
	Rahul	20

This relation does not satisfy the entity integrity constraint as here the primary key contains a NULL value.

Referential Integrity Constraints

- The Referential integrity constraints is specified between two relations or tables and used to maintain the consistency among the tuples in two relations.
- This constraint is enforced through foreign key, when an attribute in the foreign key of relation R1 have the same domain(s) as the primary key of relation R2, then the foreign key of R1 is said to reference or refer to the primary key of relation R2.
- **Example-**
- Consider the following two relations- 'Student' and 'Department'.
- Here, relation 'Student' references the relation 'Department'.



Student			Department	
<u>STU_ID</u>	Name	Dept_no	<u>Dept_no</u>	Dept_name
S001	Akshay	D10	D10	ASET
S002	Abhishek	D10	D11	ALS
S003	Shashank	D11	D12	ASFL
S004	Rahul	D14	D13	ASHS

Here,

The relation 'Student' does not satisfy the referential integrity constraint.

This is because in relation 'Department', no value of primary key specifies department no. 14.

Thus, referential integrity constraint is violated.

SQL CREATE TABLE Statement

- The CREATE TABLE statement is used to create a new table in a database.

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    column3 datatype,
    ....
);
```

- The column parameters specify the names of the columns of the table.
- The datatype parameter specifies the type of data the column can hold (e.g. varchar, integer, date, etc.).

CREATE TABLE Example

- The following example creates a table called "Persons" that contains five columns: PersonID, LastName, FirstName, Address, and City:

```
CREATE TABLE Persons (
    PersonID int,
    LastName varchar(255),
    FirstName varchar(255),
    Address varchar(255),
    City varchar(255)
);
```

Create Table Using Another Table

- A copy of an existing table can also be created using CREATE TABLE.
- The new table gets the same column definitions. All columns or specific columns can be selected.
- If you create a new table using an existing table, the new table will be filled with the existing values from the old table.

Syntax

```
CREATE TABLE new_table_name AS
    SELECT column1, column2,...
    FROM existing_table_name
    WHERE ....;
```

Example

```
CREATE TABLE TestTable AS
SELECT customername, contactname
FROM customers;
```

ALTER TABLE Statement

- The ALTER TABLE statement is used to add, delete, or modify columns in an existing table.
- The ALTER TABLE statement is also used to add and drop various constraints on an existing table.

ALTER TABLE - ADD Column

To add a column in a table, use the following syntax:

```
ALTER TABLE table_name
ADD column_name datatype;
```

- **Example**

- The following SQL adds an "Email" column to the "Customers" table:

```
ALTER TABLE Customers
ADD Email varchar(255);
```

ALTER TABLE - DROP COLUMN

- To delete a column in a table, use the following syntax:

```
ALTER TABLE table_name
DROP COLUMN column_name;
```

Example

The following SQL deletes the "Email" column from the "Customers" table:

```
ALTER TABLE Customers
DROP COLUMN Email;
```

ALTER TABLE - ALTER/MODIFY COLUMN

- To change the data type of a column in a table, use the following syntax:

SQL Server / MS Access:

```
ALTER TABLE table_name
ALTER COLUMN column_name datatype;
```

My SQL / Oracle (prior version 10G):

```
ALTER TABLE table_name
MODIFY COLUMN column_name datatype;
```

Oracle 10G and later:

```
ALTER TABLE table_name
MODIFY column_name datatype;
```

DROP TABLE Statement

- The DROP TABLE statement is used to drop an existing table in a database.
- Syntax

```
DROP TABLE table_name;
```

TRUNCATE TABLE

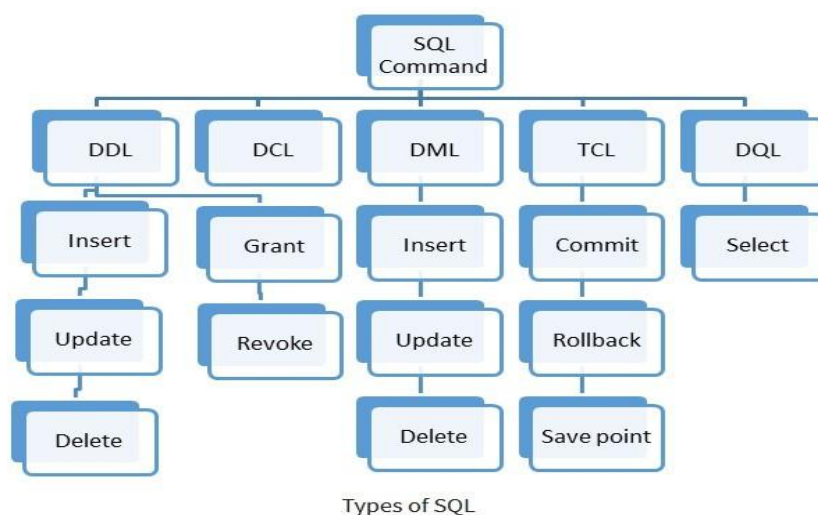
- The TRUNCATE TABLE statement is used to delete the data inside a table, but not the table itself.
- Syntax

```
TRUNCATE TABLE table_name;
```

Types of SQL

Here are five types of widely used SQL queries.

- Data Definition Language (DDL)
- Data Manipulation Language (DML)
- Data Control Language(DCL)
- Transaction Control Language(TCL)
- Data Query Language (DQL)



Different DML operations

- Data Manipulation Language (**DML**) allows you to modify the database instance by inserting, modifying, and deleting its data.
- It is responsible for performing all types of data modification in a database.
- Here are some important DML commands in SQL:
- INSERT

- UPDATE
- DELETE

INSERT

- This is a statement is a SQL query. This command is used to insert data into the row of a table.

Syntax:

```
INSERT INTO TABLE_NAME (col1, col2, col3,.... col N)
VALUES (value1, value2, value3, .... valueN);
Or
INSERT INTO TABLE_NAME
VALUES (value1, value2, value3, .... valueN);
```

For example:

```
INSERT INTO students (RollNo, FirstName, LastName) VALUES ('60', 'Tom', E
richsen');
```

UPDATE

- This command is used to update or modify the value of a column in the table.

Syntax:

```
UPDATE table_name SET [column_name1= value1,...column_nameN = valueN] [WH
ERE CONDITION]
```

For example:

```
UPDATE students
SET FirstName = 'Jhon', LastName= 'Wick'
WHERE StudID = 3;
```

DELETE

- This command is used to remove one or more rows from a table.

Syntax:

```
DELETE FROM table_name [WHERE condition];
```

For example:

```
delete from Employee where e_id=5;
```

To delete all records from the table –

```
Delete * from <TABLE NAME>;
```

-

-

-

Data Types :- Oracle supports following types of data types.

- ✚ **CHAR (SIZE)** :- The CHAR data type stores fixed-length character strings. When you create a table with a CHAR column, you must specify a string length (in bytes or characters) between 1 and 2000 bytes for the CHAR column width. The default is 1 byte.
- ✚ **VARCHAR2 (SIZE)** :- The VARCHAR2 data type stores variable-length character strings. When you create a table with a VARCHAR2 column, you specify a maximum string length (in bytes or characters) between 1 and 4000 bytes for the VARCHAR2 column.
- ✚ **VARCHAR (SIZE)** :- The VARCHAR data type is synonymous with the VARCHAR2 data type.
- ✚ **NUMBER (L)** :- The NUMBER datatype stores fixed and floating-point numbers. Numbers of virtually any magnitude can be stored and are guaranteed portable among different systems operating Oracle Database, up to 38 digits of precision.
- ✚ **NUMBER (L, D)** :- Numeric data with total number of digits L and number of digits D after decimal point.
- ✚ **DATE** :- The DATE datatype stores point-in-time values (dates and times) in a table. The DATE datatype stores the year (including the century), the month, the day, the hours, the minutes, and the seconds (after midnight).
- ✚ **LONG** :- Character data of variable length which stores up to 2 Gigabytes of data. (A bigger version of the VARCHAR2 datatype).
- ✚ **INTEGER** :- Integer type of Data. It is actually a synonym for NUMBER(38)
- ✚ **FLOAT** :- FLOAT is a 32-bit, single-precision floating-point number datatype. Each FLOAT value requires 5 bytes, including a length byte.
- ✚ **DOUBLE** :- DOUBLE is a 64-bit, double-precision floating-point number datatype. Each DOUBLE value requires 9 bytes, including a length byte.